

Fact Compression, Predictive Shielding, and Dormant Reactivation: Adaptive Memory for Sparse Embodied LLM Agents

Anonymous Group 5 Submission

Abstract

LLM agents in interactive environments must retain and prioritize information across many decision steps, yet most memory architectures are designed and benchmarked for settings where important information recurs. This leaves sparse embodied environments, where task-critical observations may appear exactly once, relatively underexplored. To address this limitation, we extend FadeMem, a decay-based memory architecture originally designed for conversational question answering, to LLM agents by introducing three mechanisms: trajectory compression into structured facts, predictive tagging that protects potentially useful facts before their importance becomes clear and reactivation of dormant memories. We evaluate six memory conditions across TextWorld and BabyAI tasks using three base LLMs. Both trajectory context and fact compression independently produce substantial gains over memoryless baselines. Dynamic memory management provides additional benefit when four conditions hold: episodes are sufficiently long, information acquisition and use is separated in time, the backbone supports reliable meta-reasoning and the agent has basic task competence.

1 Introduction

Large language model (LLM) agents are increasingly applied to sequential decision-making tasks, where they must perceive their environment and take actions based on observations accumulated over multiple steps (Yao et al., 2023; Wang et al., 2023). A central challenge in such settings is memory. Under partial observability, agents must retain past information that may be required for later planning and action (Zhang et al., 2024). Such complexities intensify within sparse embodied environments. An agent may observe a key early in an episode and only much later encounter the locked door that makes that observation relevant, after it

has already been displaced from the immediate context. Consequently, a central question emerges: *How should memory be managed in environments where task-critical information may appear once and only become useful much later?*

Existing memory strategies address this challenge only partially. Trajectory buffers preserve recent interaction history but forget older observations under a fixed budget (Zeng et al., 2024). Reflection-based approaches summarize experience across episodes rather than supporting within-episode retention (Shinn et al., 2023). Retrieval-augmented methods (Liu et al., 2024; Zhu et al., 2023) store and retrieve facts but cannot preserve information before its importance becomes clear. A more promising direction is active forgetting: FadeMem (Wei et al., 2026) introduces a biologically inspired decay mechanism over a dual memory hierarchy, with strong results on conversational benchmarks (Xu et al., 2022; Maharana et al., 2024). However, FadeMem relies on recurrence in dialogue which is absent in sparse embodied settings. In conversation, useful facts are mentioned repeatedly, which makes access frequency a reliable proxy for importance. In a sparse environment, a key may be seen once and never referred to again before it becomes necessary. Under such conditions, recurrence is no longer a reliable cue for importance.

We adapt FadeMem to sparse embodied LLM agents with three extensions: trajectory compression, predictive tagging, and dormant memory reactivation. Across three environments and three backbone LLMs, we show that both trajectory context and fact compression independently produce large gains over memoryless agents. We further show that combining them with dynamic memory management yields additional improvement only when four specific conditions are met: sufficient episode length, temporal separation between information

acquisition and use, reliable meta-reasoning, and basic agent competence.

2 Related Work

Memory Paradigms for LLM Agents. LLM agents deployed in sequential decision-making tasks must maintain relevant context across steps to support planning and goal-oriented behavior. A central challenge is determining what information to retain, how to organize it and when to retrieve it (Zhang et al., 2024; Du, 2026). Sumers et al. (2024) propose a taxonomy of working, episodic, semantic and procedural memory for language agents, which provides a useful framework for the approaches that followed. The simplest approach maintains a full trajectory in the prompt, as in ReAct (Yao et al., 2023), but fails as context grows. MemGPT (Packer et al., 2023) addresses this through virtual context management across memory tiers, but remains reactive without decay or anticipatory retention. Reflexion (Shinn et al., 2023) stores self-generated feedback across episodes without updating model weights, while ExpeL (Zhao et al., 2024) extracts generalized task rules from collected experience trajectories. Generative Agents (Park et al., 2023) go further and maintain a persistent memory stream with retrieval weighted by recency, importance and relevance, alongside periodic reflection into higher-order summaries. Together, these approaches show that structured memory improves agent behavior, but they generally assume either dense social interaction or repeated task exposure as the source of useful memory signals. We revisit that assumption below and adapt Reflexion-style episodic buffering as one of our baselines.

Structured and Cognitively-Inspired Memory. Recent work has addressed the limitations of full trajectory retention through two main strategies. Zeng et al. (2024) systematically compare memory structures such as lists, databases and knowledge graphs, and find that the best representation depends on the task. SimpleMem (Liu et al., 2026) distills interaction history into compact representations and achieves strong performance with reduced token usage on conversational benchmarks such as LoCoMo (Maharana et al., 2024) and LongMemEval-S (Wu et al., 2025). A-MEM (Xu et al., 2025) takes this further through dynamic note linking and autonomous memory evolution, with particularly strong gains on tasks requiring

multi-hop reasoning. Mem0 (Chhikara et al., 2025) provides a practical open-source memory layer for agents with automatic relevance scoring, while MemoryBank (Zhong et al., 2024) introduces an Ebbinghaus-inspired forgetting mechanism that selectively retains important information. A parallel line of work studies active forgetting. Fade-Mem (Wei et al., 2026) applies adaptive decay based on semantic relevance, access frequency and temporal signals, again drawing on Ebbinghaus’s forgetting curve, and performs strongly on conversational benchmarks including Multi-Session Chat (Xu et al., 2022) and LoCoMo (Maharana et al., 2024). Despite their differences, these approaches share an implicit assumption that important information is reinforced through recurrence or repeated access, making these signals useful proxies for importance. In sparse embodied environments, however, this assumption breaks down: task-critical observations may occur exactly once and never be revisited before they become useful.

Memory in Embodied Environments. Text based embodied environments present qualitatively different challenges for memory design than conversational settings. Benchmarks such as BabyAI (Chevalier-Boisvert et al., 2019), TextWorld (Côté et al., 2018), ALFWorld (Shridhar et al., 2021) and ScienceWorld (Wang et al., 2022) test agent capabilities in such settings. BabyAI and TextWorld capture discrete gridworld control and long horizon text interaction, while ALFWorld and ScienceWorld extend this setting to visually grounded and scientific reasoning tasks. Across these environments, rare but decision-critical events often must be acted on many steps later, a structural property absent from conversational benchmarks. Tsai et al. (2023) show that LLMs struggle with sustained planning and state tracking in text games, confirming that these environments pose genuine challenges beyond language modeling alone.

Several agent architectures have been proposed for these settings. In open-world environments, Voyager (Wang et al., 2023) addresses memory through a growing skill library in Minecraft, accumulating procedural rather than episodic knowledge and thereby sidestepping the single-observation problem at a coarser granularity. Swift-Sage (Lin et al., 2023) applies dual-process theory to interactive tasks including ScienceWorld, using a small model for routine actions and a large model

for deliberate planning. LATS (Zhou et al., 2023) unifies reasoning, acting, and planning through Monte Carlo tree search over language actions. BALROG (Paglieri et al., 2025) benchmarks agents across BabyAI, TextWorld, and four other game environments using fixed trajectory prompting, finding partial success on simpler tasks and substantial failure on harder ones requiring sustained planning. However, BALROG evaluates BabyAI mostly on single-room tasks with low memory demands, leaving open whether memory architecture becomes decisive in harder multi-room settings.

3 Methods

3.1 Fade-Enriched Memory

Our agent operates in an observe-act loop: at each step t , it receives a text observation o_t , a mission Q , and a set of valid actions, then selects an action via structured LLM generation using a hybrid prompt of recent history and retrieved facts (Section 3.1.4). To manage long-term context, we extend FadeMem (Wei et al., 2026), which introduced adaptive decay across a dual-layer memory hierarchy (SML/LML) with LLM-guided conflict resolution and semantic fusion. We adopt its core formalism for importance-weighted decay, four-way conflict classification (compatible, contradictory, subsumes, subsumed) and consolidation on access. For conflict resolution, we prepend a rule-based fast path for obvious state-change contradictions (e.g., locked/unlocked) via pattern matching with shared entity tokens; only ambiguous cases proceed to the LLM classification from (Wei et al., 2026) (details in Appendix E).

Memory Decay and Dynamics. We restate the base decay equation, as our extensions directly modulate the rate:

$$v_i(t+1) = v_i(t) \cdot \exp \left(-\lambda_{\text{base}} \cdot \xi_i \cdot e^{-\mu I_i} \cdot \text{age}_i^{\alpha_\ell} \right) \quad (1)$$

The strength $v_i \in [0, 1]$ of a fact m_i decays according to:

- λ_{base} : The base decay rate.
- I_i : An importance score combining mission relevance, access frequency and recency (Appendix A).
- α_ℓ : The layer exponent (sublinear for LML, superlinear for SML).

- ξ_i : A modifier providing decay reduction for fused facts (Appendix B).

Facts are promoted to LML when $I_i \geq 0.7$ and demoted to SML when $I_i < 0.3$.

Memory Lifecycle. Each fact transitions through three states based on its current strength v_i :

$$\text{state}(m_i) = \begin{cases} \text{active} & \text{if } v_i \geq 0.15 \\ \text{dormant} & \text{if } 0.05 \leq v_i < 0.15 \\ \text{dropped} & \text{if } v_i < 0.05 \end{cases} \quad (2)$$

Active facts are included in the agent’s prompt. Dormant facts are removed from the prompt but retained for potential reactivation (Section 3.1.3). Dropped facts (and dormant facts older than 15 steps) are permanently purged.

3.1.1 Trajectory-to-Fact Compression

To convert raw experience into retrievable knowledge, every $K = 3$ steps, a window of three observation action pairs is compressed via an LLM call into a single fact. The compression prompt follows four stages: information filtering removes mission irrelevant content, context normalization resolves spatial and temporal references for standalone interpretability, mission anchoring links the fact to the current goal Q and fact extraction produces a minimal indivisible statement. New facts are initialized with $v_i = 1.0$ in the SML and assigned an embedding vector. Each fact then immediately undergoes conflict resolution and predictive tagging (Section 3.1.2). The agent maintains a maximum of 16 active facts, with lower strength entries moved to the dormant set when this limit is exceeded.

3.1.2 Predictive Memory Tagging

We implement a prospective memory mechanism inspired by the multiprocess framework of McDaniel and Einstein (2000), which distinguishes two retrieval routes for future intentions: automatic retrieval triggered by salient cues, and monitoring that periodically scans for relevance. Our system implements both routes:

Automatic Retrieval. At fact creation, eight domain-specific regex patterns detect anticipatory associations (e.g., “picked up {color} key”). These patterns extract dynamic variables to generate specific trigger conditions (e.g., “agent encounters a locked {color} door”).

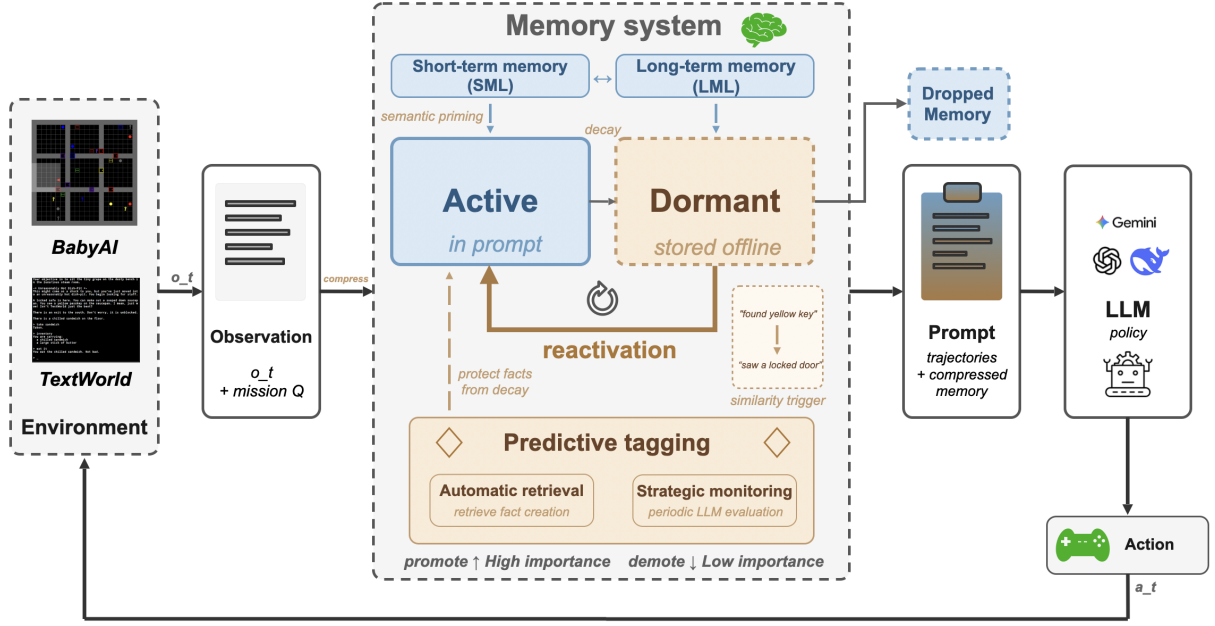


Figure 1: System architecture of our memory-augmented LLM agent. Proposed extensions for predictive tagging and reactivation appear in orange, while blue components represent the base FadeMem framework. Compression follows a four stage process (Section 3.1.1).

Strategic Monitoring. Every 12 steps, an LLM evaluates all unshielded active facts. This allows cross-fact causal reasoning to identify and assign triggers based on environmental dependencies.

Predictive Shielding. Tagged facts receive a predictive shield that caps their decay rate, effectively freezing their strength:

$$\lambda_i^{\text{shield}} = \min(\lambda_i, 0.05) \quad (3)$$

The shield persists until either (1) the trigger condition matches the current observation (similarity ≥ 0.55), upon which the shield is consumed and v_i is boosted by $+0.2$, or (2) the shield expires after 30 steps to prevent indefinite retention of stale information.

3.1.3 Dormant Memory and Reactivation

FadeMem permanently discards memories below the pruning threshold. Motivated by engram research showing that memory traces can persist but become inaccessible and be reactivated through appropriate stimulation (Josselyn and Tonegawa, 2020), we instead retain weak facts as **dormant** and implement two recovery mechanisms:

Reactivation of Dormant Facts. At each step, we compute the cosine similarity between the embeddings of dormant facts (e_i) and the current observation (e_{o_t}). If similarity exceeds $\theta_{\text{react}} = 0.25$,

the fact is reactivated:

$$m_i^{\text{active}} \leftarrow m_i^{\text{dormant}}, \quad v_i \leftarrow v_i + 0.3, \quad \text{age}_i \leftarrow 0 \quad (4)$$

This allows the environment to cue the recovery of “forgotten” knowledge. Observation embeddings are refreshed every 6 steps to reduce computation.

Semantic Priming. Weak active facts ($v_i < 0.4$) receive a strength boost when semantically related to the current observation ($\cos(e_i, e_{o_t}) > 0.5$):

$$v_i \leftarrow v_i + 0.15 \cdot \cos(e_i, e_{o_t}) \quad (5)$$

A 3-step cooldown per fact prevents artificial reinforcement loops.

3.1.4 Hybrid Prompt Architecture

The agent’s prompt combines a *trajectory buffer* (ring buffer, max 16 observation-action pairs) with a *memory block* of up to 16 active facts, ranked by a composite of mission relevance, current strength, and access frequency. Mission relevance receives the highest weight, ensuring that facts pertinent to the active goal are prioritized. Selected facts undergo consolidation on access (Wei et al., 2026), which boosts their strength to reinforce frequently retrieved information.

4 Experiments

We evaluate all memory conditions across three backbone LLMs (GPT-4o-mini, Gemini 2.5 Flash Lite, DeepSeek Chat) to assess generalizability. Each condition is evaluated over 30 episodes per difficulty level with controlled seeds and a maximum step budget of 80 steps.

4.1 Environments

TextWorld (TreasureHunter) is a text environment with procedurally generated rooms at four difficulty levels: easy (level 10), medium (level 15), hard (level 20) and very hard (level 30). These levels vary the number of rooms, object complexity, and required interactions. At each step, the agent selects from a set of admissible text commands.

BabyAI is a gridworld with discrete navigation and interaction tasks, paired with textual mission descriptions. We evaluate on four tasks with increasing memory and planning demands. *GoToObj* is a simple single-room navigation task. *Open*, *Unlock*, and *Pickup* are all set in multi-room layouts, requiring the agent respectively to open a target door, retrieve the correct key before unlocking a door, and locate a target object.

Both TreasureHunter and BabyAI are procedurally generated, preventing memorization and allowing us to test whether memory strategies scale with task complexity. We additionally evaluate on CoinCollector, a TextWorld variant without key-lock dependencies in which the agent collects coins in procedurally generated rooms (Level 10; results are reported in Appendix L).

4.2 Memory Conditions

We compare our architecture against five memory strategies spanning the design space of agent context management, following a two-tiered structure.

Tier 1: Baseline comparisons. Each baseline answers a distinct design question:

- *Baseline* provides the current observation.
- *Trajectory* provides the full chat history (max 16 pairs).
- *Reflection* adds cross-episode learning via structured self-reflections in the system prompt but has no within-episode memory.
- *Enriched* compresses every 3 steps into a fact (FIFO, max 16) without chat history.

Tier 2: Controlled ablation.

- *Enriched History* combines trajectory and compressed facts with the same prompt structure as our full system, but without any forgetting, tagging, or reactivation mechanisms.
- *Fade-Enriched History* is our full system, which adds dynamic memory mechanisms on top of *Enriched History*.

Table 1 summarizes all memory conditions.

Condition	Description
Baseline	Current observation only, no memory
Trajectory Reflection	Full chat history (max 16 pairs) No within-episode memory; cross-episode self-reflections in system prompt
Enriched	Compressed facts (FIFO, max 16), no chat history
Enriched History	Chat history + compressed facts (FIFO)
Fade-Enriched History	Chat history + compressed facts with dynamic memory management (decay, reactivation, priming, conflict resolution, fusion, predictive tagging)

Table 1: Overview of memory conditions.

4.3 Evaluation Metrics

Success Rate (SR) is our primary metric: the fraction of episodes where the agent completed the mission (terminated with reward > 0). We report 95% confidence intervals via the Wilson score method. We also report average steps to success, loop rate, and blocked forward rate (BabyAI only); formal definitions are provided in Appendix O.

5 Results and Discussion

5.1 TextWorld Results

Figure 2 presents success rates with 95% confidence intervals across all four difficulty levels and three models. The pattern is fairly consistent across all panels. Baseline and Reflection both sit at the bottom, rarely getting above 40% regardless of difficulty, and their bars are close enough that there is almost no real difference between them. This indicates that giving the agent lessons from past episodes does not actually help when layouts keep changing.

There is a clear jump from these two up to Enriched memory, which performs better across the



Figure 2: Success rates (%) on TextWorld Treasure-Hunter across difficulty levels L10, L15, L20, and L30. Panels correspond to GPT-4o-mini (GPT), Gemini-2.5-Flash-Lite (GEM) and DeepSeek Chat (DS). Graph includes Wilson 95% confidence intervals and corresponding interval values are also reported in Appendix H.

board. This shows that fact compression gives the agent useful long-term context. Both Enriched and Trajectory independently produce substantial improvements over Baseline and Reflection, with Trajectory consistently outperforming Enriched. This indicates that raw sequential context is the stronger signal, while fact compression provides complementary value. Combining both (Enriched History) performs comparably to Trajectory alone, with dynamic memory management providing additional benefit on harder tasks where the FIFO buffer becomes insufficient for managing competing information.

Trajectory, Enriched History, and Fade-Enriched History form the top tier. At L10 and L15 their bars are close and confidence intervals broadly overlap, so there is little to separate them at easier difficulties. The more interesting differences show up at L20 and L30. Exact numbers for all conditions and

models are reported in Appendix I.

Hard (Level 20). Fade-Enriched History performs best at this level and the gap is most visible in Figure 2 where its bars sit above Enriched History and Trajectory, particularly for Gemini and GPT. The advantage is clearest for Gemini (+4pp) and GPT (+7pp), while DeepSeek saturates at 97% across all top-tier conditions, leaving no room for differentiation. Moreover, Fade achieves this higher success rates with comparable steps to Enriched History (See Table 8 in Appendix), indicating that the additional meta-reasoning calls do not slow down decision-making in terms of environment interaction.

Very Hard (Level 30). Success rates here (Figure 2, Table 6) reveal critical model-dependent trade-offs. While Gemini 2.5 maintains a clear advantage using Fade over Enriched History (83% vs. 77% in Table 6), GPT-4o-mini suffers a significant drop to 37%. Notably, DeepSeek Chat achieves its peak performance of 97% with uncompressed Trajectory (Table 6). Its decline to 87% under the Fade condition (Table 6) indicates that for top-tier backbones, explicit fact compression and meta-reasoning calls can introduce detrimental overhead or information loss. Detailed evaluation traces in Appendix M.3 suggest that for high-capacity models, raw sequential history remains the most reliable signal, whereas automated memory management might prune subtle but necessary environmental cues. As further analyzed in Appendix M.4, weaker models like GPT-4o-mini can actively destroy their own memory strength through cascading conflict misclassifications during these long-horizon episodes.

5.2 BabyAI Results

Condition	GoToObj	Open	Pick	Unlock
Baseline	13.3	8.9	1.1	1.1
Reflection	13.3	8.9	1.1	3.3
Enriched	36.7	13.3	5.6	5.6
Trajectory	51.1	15.6	15.6	5.6
Enriched Hist.	61.1	23.3	12.2	12.2
Fade-Enr. Hist.	56.7	18.9	15.6	6.7

Table 2: Mean success rates (%) on BabyAI, averaged across three backbone LLMs. Bold indicates best per column. Per-model results in Appendix.

Table 2 presents mean success rates on BabyAI, averaged across three backbone LLMs. Task complexity increases sharply from GoToObj (simple

navigation) through Open and Pickup (requiring object interaction) to Unlock (requiring a multi-step key-door sequence), which is reflected in the decreasing success rates across all conditions.

The dominant failure mode is behavioral looping (See Table 9 in Appendix): loop rates range from 43% to 100% across conditions and tasks. The high Blocked Forward Rates (often 40–95%) confirm that the primary bottleneck is spatial reasoning rather than memory management.

Regarding BabyAI GoToObj results (Table 2), memory strategies produce a clear performance gain over the reactive Baseline. Both Enriched History (61.1%) and Fade-Enriched History (56.7%) significantly outperform the memoryless agent (13.3%). However, DeepSeek Chat performs best on this task using Trajectory alone (80.0%), outperforming our full Fade-Enriched system by 23 percentage points (Table 7). High loop rates of 60–100% across all tasks (Table 9) imply that the primary bottleneck remains spatial reasoning rather than memory capacity. Even with access to a summary of prior actions, agents often fail due to an inability to translate diagonal spatial descriptions into efficient action sequences, as documented in the reasoning traces of Appendix M.2.

5.3 Controlled Ablation

The comparison between Enriched History and Fade-Enriched History directly isolates the contribution of our dynamic memory management (full results in Appendix I). On TextWorld, Fade outperforms Enriched History on easy (+6.7 percentage points) and hard L20 (+3.3pp), while matching on medium and slightly trailing on very hard L30 (−2.2pp). On BabyAI, Fade leads on Pickup (+3.3pp) but trails on GoToObj (−4.4pp) and Unlock (−5.6pp).

Two factors explain the inconsistency: short episodes where the FIFO buffer never overflows and the periodic mechanisms never activate (see Condition 1 below), and weaker backbones where the meta-reasoning calls may introduce errors (see Condition 3; Appendix M for a representative example). This explains why Fade underperforms Enriched History with GPT-4o-mini on very hard L30 (37% vs. 43%) while outperforming it with Gemini (83% vs. 77%).

With 30 episodes, the confidence intervals are wide. The patterns we discuss are nonetheless consistent across backbones and difficulty levels.

5.4 Qualitative Analysis

We provide qualitative analysis of individual reasoning traces in Appendix M, including a demonstration of predictive tagging in action on TextWorld and an analysis of the loop failure mode on BabyAI. The annotated traces provide mechanical causes behind the performance patterns observed in the quantitative results.

Prospective Memory and Shielding. Traces on TextWorld L30 (Appendix M.1) demonstrate the effectiveness of predictive tagging for models in the moderate capacity range. In several episodes, agents acquired keys up to 18 steps before encountering the corresponding lock. Whereas standard FIFO displaced these facts with transient navigation information, Fade’s predictive shield preserved the key fact in the prompt, enabling successful task completion.

Cascading Meta-Reasoning Failures. Analysis of GPT-4o-mini traces (Appendix M.4) helps explain its poor performance on more complex levels. The model systematically misclassifies compatible navigational updates as contradictory. Each such error applies an exponential decay penalty, averaging -0.240 in v_i , which progressively removes valid knowledge from active memory. In this regime, passive buffers can outperform Fade because they do not actively suppress information that is still correct and potentially useful.

Spatial Reasoning Barrier. Reasoning traces from BabyAI failures (Appendix M.2) show that memory architecture remains secondary to basic navigation competence. Agents frequently enter oscillation loops, repeatedly turning between directions without attempting forward movement even when the target remains visible. These traces confirm that improved memory cannot compensate for the absence of fundamental spatial reasoning in text-only models.

5.5 Discussion

Our results suggest that dynamic memory management helps under a specific set of conditions and is ineffective or counterproductive when any of them is absent. We identify four necessary conditions and show how each environment in our evaluation maps onto them.

Condition 1: Sufficient episode length. Fade mechanisms require time to provide value beyond

simple buffers. Average steps to success for GPT on Level 10 (Table 10) range between 4-8 steps. Since many of these episodes resolve before the 12-step periodic tagging cycle activates (Table 3), Fade provides limited additional benefit over Enriched History in short horizons.

Condition 2: Temporal separation between information acquisition and use. The predictive tagging mechanism is designed for situations where information acquired early becomes critical only much later. CoinCollector, which shares TextWorld’s text interface but lacks such temporal dependencies (the agent simply collects coins), confirms this. On Level 10, Fade achieves 94% vs. Enriched History’s 88% mean SR across backbones, with no advantage on any individual LLM and low trigger activations (full results in Appendix L). This is expected: without such delayed dependencies, predictive shielding has little opportunity to improve performance.

Condition 3: Backbone Capacity. Dynamic memory management occupies a sweet spot, assisting models capable of reliable meta-reasoning while introducing noise or overhead for those outside this range. Weak backbones such as GPT-4o-mini suffer from meta-calls that actively degrade memory quality. Appendix M.4 shows that systematic misclassification of compatible facts as contradictory applies severe decay penalties, averaging -0.240 in v_i , which progressively purges valid knowledge from active memory. This helps explain why GPT-4o-mini drops to 37% with Fade compared to 43% with Enriched History on L30 (Table 6). Conversely, high-capacity models such as DeepSeek incur an overhead penalty, as uncompressed trajectory context remains the most reliable signal. On L30, DeepSeek reaches 97% with Trajectory alone but declines to 87% with Fade (Table 6). Appendix M.3 suggests that compression may prune subtle environmental cues that such models can otherwise exploit directly from sequential history. Gemini occupies the middle ground, showing the clearest advantage for Fade on L30 (83% vs. 77% for Enriched History in Table 6) by combining sufficient reasoning reliability with benefit from structured memory management.

Condition 4: Basic task competence. The agent must be able to perform the underlying task before memory can make a difference. On BabyAI, memory strategies produce clear gains on GoToObj

(the simplest task), but on advanced tasks loop rates of 60–100% across all conditions indicate that spatial reasoning failures compound to the point where no memory architecture can compensate. Although these tasks contain key-door dependencies that would benefit from memory, the agent rarely progresses far enough for memory to matter.

Where all conditions are met. TextWorld hard L20 represents the regime where all four conditions hold: episodes are long enough, the task contains key-lock dependencies, and the backbone handles meta-calls reliably. Here, Fade matches or outperforms Enriched History across all three models, with the clearest gains on Gemini and GPT. More broadly, dynamic memory management occupies a sweet spot: it benefits models capable enough to execute meta-reasoning reliably, but not so strong that raw context alone suffices.

6 Conclusion and Future Work

We extended FadeMem from conversational QA to interactive LLM agents, introducing trajectory compression, predictive tagging inspired by prospective memory (McDaniel and Einstein, 2000) and dormant memory reactivation based on silent engram theory from neuroscience (Josselyn and Tonegawa, 2020). Both trajectory context and fact compression independently produce substantial gains over memoryless agents. Through systematic evaluation across three environments, three base LLMs and six memory conditions, we identified four conditions under which dynamic memory management provides additional benefit: sufficient episode length, temporal separation between acquisition and use, a backbone in the right capacity range for meta-reasoning and basic task competence.

Our evaluation has several limitations. We test Fade as a complete system without component-level ablations. The sample size of 30 episodes per condition, chosen due to computational costs, produces wide confidence intervals. The three base models are all relatively lightweight. Stronger models (e.g., GPT-4o, Claude) may shift the sweet spot identified in Condition 3. BabyAI evaluation is limited by navigation incompetence of text-only LLMs rather than by memory. Future work should address these gaps through individual mechanism ablations, stronger backbones, visual input for navigation tasks and environments with longer episodes and richer temporal dependencies (e.g., TextWorld Cooking, NetHack, ScienceWorld).

References

- Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. 2019. BabyAI: A platform to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Ruo Yu Tao, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. 2018. TextWorld: A learning environment for text-based games.
- Pengfei Du. 2026. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*.
- Sheena A. Josselyn and Susumu Tonegawa. 2020. Memory engrams: Recalling the past and imagining the future. *Science*, 367(6473).
- Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. 2023. SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks. In *Advances in Neural Information Processing Systems*.
- Jiaqi Liu et al. 2026. Simplemem: Efficient life-long memory for llm agents. *arXiv preprint arXiv:2601.02553*.
- Na Liu, Liangyu Chen, Xiaoyu Tian, Wei Zou, Kaijiang Chen, and Ming Cui. 2024. From LLM to conversational agent: A memory enhanced architecture with fine-tuning of large language models.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Mark A. McDaniel and Gilles O. Einstein. 2000. Strategic and automatic processes in prospective memory retrieval: A multiprocess framework. *Applied Cognitive Psychology*, 14:S127–S144.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wołczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. 2025. Balrog: Benchmarking agentic llm and vlm reasoning on games. In *International Conference on Learning Representations (ICLR)*.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulators of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. Cognitive architectures for language agents. *Transactions on Machine Learning Research*.
- Chen Feng Tsai, Xiaochen Zhou, Sierra S. Liu, Jing Li, Mo Yu, and Hongyuan Mei. 2023. Can large language models play text games well? Current state-of-the-art and open questions. *arXiv preprint arXiv:2304.02868*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.
- Lei Wei et al. 2026. Fademem: Biologically-inspired forgetting for efficient agent memory. *arXiv preprint arXiv:2601.18642*.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*.
- Jing Xu, Arthur Szlam, and Jason Weston. 2022. Beyond goldfish memory: Long-term open-domain conversation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5180–5197.

- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations (ICLR)*.
- Ruihong Zeng, Jinyuan Fang, Siwei Liu, and Zaiqiao Meng. 2024. [On the structural memory of LLM agents](#).
- Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Jirong Wen. 2024. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*.
- Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, Yu Qiao, Zhaoxiang Zhang, and Jifeng Dai. 2023. [Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory](#).

950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

A Importance Score

The importance score I_i from Eq. 1 is defined as:

$$I_i = \alpha \cdot \text{rel}(m_i, Q) + \beta \cdot \frac{\tilde{f}_i}{1 + \tilde{f}_i} + \gamma \cdot \exp(-\delta \cdot \text{age}_i) \quad (6)$$

where $\text{rel}(m_i, Q)$ is the cosine similarity between the fact embedding and the mission embedding, $\tilde{f}_i = \sum_j \exp(-\kappa(t - t_j))$ is a time-weighted access count over the steps t_j at which the fact was selected for the prompt (with $\kappa = 0.1$), and age_i counts the steps since m_i was last accessed (with $\delta = 0.1$). The weights are $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$. The time-weighted access count gives more credit to recent accesses, modeling the recency aspect of memory strength.

B Fusion Details

The fusion decay modifier $\xi_i = 1/(1 + \log |C_i|)$ reduces the decay rate for facts that were fused from $|C_i|$ source facts during periodic memory consolidation ($\xi_i = 1$ for unfused facts). Fusion proceeds as follows:

Clustering. Greedy single-linkage clustering groups facts with $\cos(\mathbf{e}_i, \mathbf{e}_j) > 0.82$ and $|\text{step}_i - \text{step}_j| < 10$.

Merging. The LLM generates a merged fact from each cluster. The fused fact receives strength $v_{\text{fused}} = \max_i(v_i) + 0.1 \cdot \text{Var}(v_i)$.

Validation. The LLM scores information preservation $\in [0, 1]$. If the score falls below $\theta_{\text{preserve}} = 0.7$, the fusion is rejected and the original facts are retained.

Fusion runs every 12 steps alongside batch tagging.

C Trigger Matching

The trigger matching score determines whether a shielded fact’s trigger condition is met by the current observation:

$$\text{match}(\text{trigger}, o_t) = \frac{|\text{tok}(\text{trigger}) \cap \text{tok}(o_t)|}{|\text{tok}(\text{trigger})|}$$

where $\text{tok}(\cdot)$ produces stopword-filtered lowercase tokens. A match is registered when this score exceeds 0.55. When embedding similarity is available, it is used as an alternative: $\cos(\mathbf{e}_{\text{trigger}}, \mathbf{e}_{o_t}) \geq 0.55$.

D Token-Overlap Fallback

When embeddings are unavailable (e.g., due to API failure or rate limiting), the system falls back to a token-overlap relevance score:

$$\text{rel}(m_i, Q, o_t) = 0.7 \cdot \frac{|\text{tok}(m_i) \cap \text{tok}(Q)|}{|\text{tok}(Q)|} + 0.3 \cdot \frac{|\text{tok}(m_i) \cap \text{tok}(o_t)|}{|\text{tok}(o_t)|}$$

Mission overlap is weighted higher (0.7) because the goal remains stable across steps while observations change constantly, making mission alignment a more reliable relevance signal.

E Conflict Resolution Details

Rule-based fast path. Eight antonym patterns are checked with shared entity words (e.g., “the blue door is open” vs. “the blue door is closed”). On match, the older fact receives $v_i *= 0.5$ and transitions to dormant.

Embedding + LLM classification. Active facts with $\cos(\mathbf{e}_i, \mathbf{e}_{\text{new}}) > 0.65$ are classified by the LLM into one of four categories:

Relation	Action
Compatible	$v_i *= (1 - 0.4 \cdot \text{sim})$
Contradictory	$v_i *= \exp(-\rho \cdot \text{norm_age})$
Subsumes	Old $\rightarrow$ dormant; new receives LLM-merged text
Subsumed	New $\rightarrow$ dormant; old receives merge + boost

The parameter $\rho = 1.0$ controls the suppression strength and $\text{norm_age} = \min\left(1, \frac{\text{current_step} - \text{creation_step}}{W_{\text{step}}}\right)$ with $W_{\text{step}} = 30$.

Redundancy penalty for compatible facts.

When a new fact is added, existing facts with cosine similarity above a threshold $\theta_{\text{sim}} = 0.65$ are retrieved and classified as compatible, contradictory, subsumes, or subsumed.

For compatible pairs, both facts can be true simultaneously. To discourage redundant accumulation of highly similar facts, the older fact’s strength is reduced via a multiplicative penalty:

$$v_i *= (1 - \omega \cdot \text{sim}), \quad (7)$$

where $\omega = 0.4$ is the redundancy penalty weight and sim is the cosine similarity between fact embeddings. This gradually down-weights overlapping facts while retaining them if no better alternative exists.

F Predictive Tagging Patterns

The eight regex patterns for rule-based predictive tagging:

1. “picked up {color} key” → “locked {color} door”
2. “found {color} key” → “locked {color} door”
3. “took {object}” → “needs {object}”
4. “locked {color} door” → “{color} key”
5. “closed {color} chest” → “open {color} chest”
6. “blocked path {direction}” → “clear path {direction}”
7. “discovered {location}” → “return to {location}”
8. “inventory contains {item}” → “use {item}”

Patterns extract dynamic variables (color, object type, direction, location) via named capture groups to generate specific trigger conditions.

G Hyperparameters

Table 3 reports the hyperparameters used for the experiments.

H Wilson 95% Confidence Intervals

Tables 4–5 report Wilson 95% confidence intervals for all TextWorld and BabyAI tasks.

I Per-Model TextWorld Success Rates

Table 6 reports the full success rates of all TextWorld tasks across different backbone LLMs.

J Per-Model BabyAI Success Rates

Table 7 reports the full success rates of all babyAI tasks across different backbone LLMs.

K Additional TextWorld and BabyAI Diagnostics

Table 8 – 9 provide additional diagnostic results for TextWorld and BabyAI, averaged over all tasks within each environment. We report success rate, loop rate, average steps across all episodes, and average steps to success.

Table 10 provides the corresponding per-level diagnostic results for GPT-4o-mini on TextWorld TreasureHunter.

Table 11 provides loop rates by BabyAI task and memory condition, averaged across all backbone LLMs.

L CoinCollector Results

CoinCollector is a TextWorld variant where the agent collects coins in procedurally generated rooms without key-lock dependencies. We evaluate on Level 10 (30 episodes, max 20 steps) to test whether Fade mechanisms provide benefit in the absence of temporal separation between information acquisition and use (Condition 2 in Section 5.5).

M Reasoning Traces

M.1 Predictive Tagging in Action

The following trace in Table 20 demonstrates the full lifecycle of three core mechanisms in a single 44-step episode on TextWorld TreasureHunter very hard, Level 30 (Gemini 2.5 Flash Lite, fade.enriched.history).

Analysis. The trace demonstrates the full lifecycle of three core mechanisms in a single 44-step episode on TextWorld TreasureHunter very hard, Level 30.

Predictive Tagging. At Step 2, the agent picks up the type 3 latchkey. The Batch Tagging module matches the pattern “picked up.*key” and creates a shield on the corresponding fact with trigger condition `locked.*(chest|door|passageway)`. This implements prospective memory: at encoding time, the system anticipates a future retrieval context and sets a condition for it. A second shield is created at Step 22 when the agent retrieves a latchkey from the type 3 chest, with trigger condition `locked.*(door|passage)`.

Decay and the critical threshold. Between Steps 2 and 19, the agent explores 10 rooms without encountering any locked object. The fact “Agent picked up type 3 latchkey in bedroom” decays steadily from $v_i = 0.967$ to $v_i = 0.271$. At Step 19, the fact has decayed substantially to $v_i = 0.271$, approaching the active-to-dormant boundary at 0.15. Without the shield mechanism, continued exploration would further decay v_i , eventually pushing it below the active threshold and removing it from the prompt. The agent would then encounter the locked chest at Step 20 without any memory of possessing a key.

In some episodes, we observe temporal gaps of up to 18 steps between acquisition and use. Such long delays are precisely the regime in which predictive tagging becomes useful, because the relevant fact must survive extended exploration be-

Parameter	Value	Description
K	3	Extraction window size (steps)
Buffer size	16	Maximum active facts
α, β, γ	0.5, 0.3, 0.2	Importance weights
δ	0.1	Recency decay rate
κ	0.1	Access count decay rate
λ_{base}	0.1	Base decay rate
μ	3.0	Importance-decay coupling
α_{SML}	1.2	SML age exponent
α_{LML}	0.8	LML age exponent
θ_{promote}	0.7	SML $\rightarrow$ LML threshold
θ_{demote}	0.3	LML $\rightarrow$ SML threshold
θ_{dormant}	0.15	Active $\rightarrow$ dormant threshold
θ_{drop}	0.05	Dormant $\rightarrow$ dropped threshold
Δv	0.2	Consolidation boost
N	5	Consolidation diminishing rate
θ_{sim}	0.65	Conflict candidate threshold
θ_{fusion}	0.82	Fusion similarity threshold
T_{window}	10	Fusion temporal window (steps)
θ_{preserve}	0.7	Fusion validation threshold
λ_{shield}	0.05	Shielded decay cap
Shield timeout	30	Max shield duration (steps)
θ_{react}	0.25	Reactivation threshold
θ_{prime}	0.5	Priming threshold
Priming cooldown	3	Steps between priming events
Priming boost	0.15	Max priming strength boost
Reactivation boost	0.3	Strength boost on reactivation
Trigger boost	0.2	Strength boost on trigger match
Fusion/tagging interval	12	Steps between periodic phases
Obs. embedding refresh	6	Steps between re-embedding
Dormant max age	15	Max steps before permanent drop

Table 3: Hyperparameters for the experiments

fore the corresponding trigger appears. The longer episode length is precisely what makes predictive tagging necessary: on Level 10, where episodes resolve before the periodic mechanisms activate (Condition 1, Section 5.5), Fade and Enriched History perform nearly identically.

Trigger firing. At Step 20, the agent’s observation contains “locked type 3 chest,” which matches the trigger condition `locked.*chest`. The shield is consumed and v_i receives a +0.2 boost from 0.271 to 0.471, well above the active threshold. The fact remains in the agent’s prompt, and at Step 21 the agent successfully unlocks the chest. This mechanism contributes to Fade’s advantage over Enriched History on Level 30 with Gemini (83% vs. 77%, Table 6), where the FIFO buffer of Enriched History has no equivalent way to preserve facts across extended exploration. By contrast, on CoinCollector (Table 12), which lacks key-lock dependencies and therefore never activates trigger conditions, Fade shows no advantage (Condition 2, Section 5.5).

Reactivation. At Step 36, a previously dormant navigation fact ($v_i = 0.147$, below the active threshold) is reactivated to $v_i = 0.447$ when

the agent returns to a contextually similar location. This demonstrates Consolidation on Access: dormant facts are not permanently lost but persist as silent engrams that can be retrieved when the current observation provides a sufficiently similar context, consistent with the memory reactivation framework of Josselyn and Tonegawa (2020).

Counterfactual. Without Predictive Tagging, the fact “picked up type 3 latchkey” would have continued to decay and eventually transitioned to the dormant state ($v_i < 0.15$), risking exclusion from the prompt at a critical decision point. The agent would have encountered the locked chest but failed to recall the key. The 18 steps of intervening exploration between key acquisition (Step 2) and key use (Step 21) represent exactly the kind of temporal separation that causes information loss under standard exponential decay, and exactly the scenario that Predictive Tagging is designed to address. Notably, this failure mode does not appear when the backbone is strong enough that raw context suffices without compression: DeepSeek achieves 97% with Trajectory alone on Level 30 (Table 6), bypassing the need for dynamic memory entirely. This trace therefore also illustrates Condition 3

Model	Memory Type	L10	L15	L20	L30
GPT	baseline	23.3% [11.8%, 40.9%]	16.7% [7.3%, 33.6%]	16.7% [7.3%, 33.6%]	16.7% [7.3%, 33.6%]
	enriched	56.7% [39.2%, 72.6%]	56.7% [39.2%, 72.6%]	30.0% [16.7%, 47.9%]	23.3% [11.8%, 40.9%]
	enriched_history	80.0% [62.7%, 90.5%]	100.0% [88.6%, 100.0%]	63.3% [45.5%, 78.1%]	43.3% [27.4%, 60.8%]
	fade_enriched_history	93.3% [78.7%, 98.2%]	93.3% [78.7%, 98.2%]	70.0% [52.1%, 83.3%]	36.7% [21.9%, 54.5%]
	reflection	26.7% [14.2%, 44.4%]	13.3% [5.3%, 29.7%]	23.3% [11.8%, 40.9%]	20.0% [9.5%, 37.3%]
	trajectory	86.7% [70.3%, 94.7%]	93.3% [78.7%, 98.2%]	66.7% [48.8%, 80.8%]	46.7% [30.2%, 63.9%]
GEM	baseline	30.0% [16.7%, 47.9%]	10.0% [3.5%, 25.6%]	23.3% [11.8%, 40.9%]	20.0% [9.5%, 37.3%]
	enriched	73.3% [55.6%, 85.8%]	63.3% [45.5%, 78.1%]	66.7% [48.8%, 80.8%]	33.3% [19.2%, 51.2%]
	enriched_history	86.7% [70.3%, 94.7%]	93.3% [78.7%, 98.2%]	93.3% [78.7%, 98.2%]	76.7% [59.1%, 88.2%]
	fade_enriched_history	90.0% [74.4%, 96.5%]	96.7% [83.3%, 99.4%]	96.7% [83.3%, 99.4%]	83.3% [66.4%, 92.7%]
	reflection	40.0% [24.6%, 57.7%]	23.3% [11.8%, 40.9%]	26.7% [14.2%, 44.4%]	26.7% [14.2%, 44.4%]
	trajectory	83.3% [66.4%, 92.7%]	93.3% [78.7%, 98.2%]	86.7% [70.3%, 94.7%]	76.7% [59.1%, 88.2%]
DS	baseline	43.3% [27.4%, 60.8%]	26.7% [14.2%, 44.4%]	36.7% [21.9%, 54.5%]	33.3% [19.2%, 51.2%]
	enriched	83.3% [66.4%, 92.7%]	83.3% [66.4%, 92.7%]	66.7% [48.8%, 80.8%]	50.0% [33.2%, 66.8%]
	enriched_history	96.7% [83.3%, 99.4%]	100.0% [88.6%, 100.0%]	96.7% [83.3%, 99.4%]	93.3% [78.7%, 98.2%]
	fade_enriched_history	100.0% [88.6%, 100.0%]	100.0% [88.6%, 100.0%]	96.7% [83.3%, 99.4%]	86.7% [70.3%, 94.7%]
	reflection	56.7% [39.2%, 72.6%]	26.7% [14.2%, 44.4%]	33.3% [19.2%, 51.2%]	30.0% [16.7%, 47.9%]
	trajectory	96.7% [83.3%, 99.4%]	100.0% [88.6%, 100.0%]	96.7% [83.3%, 99.4%]	96.7% [83.3%, 99.4%]

Table 4: TextWorld results across three backbones (DS = DeepSeek Chat, GEM = Gemini 2.5 Flash Lite, GPT = GPT-4o-mini.). Each entry reports success rate with Wilson 95% confidence interval across 30 episodes.

(Section 5.5): Fade provides the clearest benefit for backbones in the middle capacity range, where raw context is insufficient but meta-reasoning calls execute reliably.

M.2 Loop Failure on BabyAI GoToObj

Tables 13 and 14 show representative loop failures from BabyAI GoToObj under two different memory conditions.

Analysis. In Table 13 (Trajectory, Seed 0), the agent observes the green key at a diagonal position: “1 step right and 2 steps forward.” Because the key is not directly ahead, the agent turns right to face it. After turning, the key appears at “2 steps left and 1 step forward,” so the agent turns left. This restores the original observation, and the cycle repeats for all 64 steps until truncation. The agent never selects `go forward` despite it being available at every step.

The LLM reasoning is superficially plausible at each step (“I need to turn right to face the direction of the green key”), but reflects a fundamental inability to decompose diagonal movement into sequential actions: moving forward first, then turning.

Table 14 (Enriched History, Seed 1) shows the same failure pattern under a condition that provides the agent with a summary of its prior actions. Despite having access to its action history, the agent enters the same oscillation between `turn left` and `turn right` for 64 steps without recovery. The enriched prompt does not help the LLM recog-

nize the repetition or try an alternative action.

This pattern is not an isolated case. As Table 9 shows, loop rates across all BabyAI tasks remain high regardless of memory condition or backbone: 75.0–82.5% for GPT-4o-mini, 70.0–90.8% for Gemini, and 75.0–90.8% for DeepSeek. Even the best-performing condition (Enriched History with Gemini, 70.0% loop rate) still produces loops in the majority of episodes. Crucially, adding Fade memory on top of Enriched History does not reduce loop rates: GPT-4o-mini increases from 75.0% to 80.0%, Gemini from 70.0% to 75.0%, and DeepSeek from 81.7% to 75.8%. The corresponding success rates remain low across all conditions, peaking at 35.0% (DeepSeek, Trajectory) and never exceeding 30.0% for any memory-augmented condition.

These results confirm that the bottleneck on BabyAI is spatial navigation competence (Condition 4) rather than memory capacity. The agents fail not because they forget prior observations, but because they cannot translate relative spatial descriptions into correct action sequences. No memory architecture can resolve a failure where the agent never attempts forward movement.

M.3 Fade vs. Enriched History on Same Seed

The following comparison in Tables 15 and 16 shows the same mission (TextWorld Treasure-Hunter very hard, Level 30, Gemini 2.5 Flash Lite, Seed 1) under two memory conditions: Fade-Enriched History and Enriched History. Both conditions see identical room layouts and object place-

Model	Memory Type	GoToObj	Open	Pickup	Unlock
GPT	baseline	13.3% [5.3%, 29.7%]	10.0% [3.5%, 25.6%]	3.3% [0.6%, 16.7%]	0.0% [0.0%, 11.4%]
	enriched	13.3% [5.3%, 29.7%]	10.0% [3.5%, 25.6%]	6.7% [1.8%, 21.3%]	0.0% [0.0%, 11.4%]
	enriched_history	50.0% [33.2%, 66.8%]	16.7% [7.3%, 33.6%]	10.0% [3.5%, 25.6%]	13.3% [5.3%, 29.7%]
	fade.enriched_history	53.3% [36.1%, 69.8%]	13.3% [5.3%, 29.7%]	13.3% [5.3%, 29.7%]	3.3% [0.6%, 16.7%]
	reflection	13.3% [5.3%, 29.7%]	16.7% [7.3%, 33.6%]	3.3% [0.6%, 16.7%]	10.0% [3.5%, 25.6%]
	trajectory	36.7% [21.9%, 54.5%]	3.3% [0.6%, 16.7%]	10.0% [3.5%, 25.6%]	0.0% [0.0%, 11.4%]
GEM	baseline	13.3% [5.3%, 29.7%]	10.0% [3.5%, 25.6%]	0.0% [0.0%, 11.4%]	3.3% [0.6%, 16.7%]
	enriched	83.3% [66.4%, 92.7%]	20.0% [9.5%, 37.3%]	6.7% [1.8%, 21.3%]	10.0% [3.5%, 25.6%]
	enriched_history	63.3% [45.5%, 78.1%]	30.0% [16.7%, 47.9%]	10.0% [3.5%, 25.6%]	13.3% [5.3%, 29.7%]
	fade.enriched_history	60.0% [42.3%, 75.4%]	23.3% [11.8%, 40.9%]	23.3% [11.8%, 40.9%]	10.0% [3.5%, 25.6%]
	reflection	13.3% [5.3%, 29.7%]	10.0% [3.5%, 25.6%]	0.0% [0.0%, 11.4%]	0.0% [0.0%, 11.4%]
	trajectory	36.7% [21.9%, 54.5%]	13.3% [5.3%, 29.7%]	13.3% [5.3%, 29.7%]	10.0% [3.5%, 25.6%]
DS	baseline	13.3% [5.3%, 29.7%]	6.7% [1.8%, 21.3%]	0.0% [0.0%, 11.4%]	0.0% [0.0%, 11.4%]
	enriched	13.3% [5.3%, 29.7%]	10.0% [3.5%, 25.6%]	3.3% [0.6%, 16.7%]	6.7% [1.8%, 21.3%]
	enriched_history	70.0% [52.1%, 83.3%]	23.3% [11.8%, 40.9%]	16.7% [7.3%, 33.6%]	10.0% [3.5%, 25.6%]
	fade.enriched_history	56.7% [39.2%, 72.6%]	20.0% [9.5%, 37.3%]	10.0% [3.5%, 25.6%]	6.7% [1.8%, 21.3%]
	reflection	13.3% [5.3%, 29.7%]	0.0% [0.0%, 11.4%]	0.0% [0.0%, 11.4%]	0.0% [0.0%, 11.4%]
	trajectory	80.0% [62.7%, 90.5%]	30.0% [16.7%, 47.9%]	23.3% [11.8%, 40.9%]	6.7% [1.8%, 21.3%]

Table 5: BabyAI results across all three backbones (DS = DeepSeek Chat, GEM = Gemini 2.5 Flash Lite, GPT = GPT-4o-mini.). Each entry reports success rate with Wilson 95% confidence interval across 30 episodes.

Condition	Easy (L10)			Medium (L15)			Hard (L20)			Very Hard (L30)		
	DS	GEM	GPT	DS	GEM	GPT	DS	GEM	GPT	DS	GEM	GPT
Baseline	43	30	23	27	10	17	37	23	17	33	20	17
Reflection	57	40	27	27	23	13	33	27	23	30	27	20
Enriched	83	73	57	83	63	57	67	67	30	50	33	23
Trajectory	97	83	87	100	93	93	97	87	67	97	77	47
Enriched Hist.	97	87	80	100	93	100	97	93	63	93	77	43
Fade-Enr. Hist.	100	90	93	100	97	93	97	97	70	87	83	37

Table 6: Success rates (%) on TextWorld TreasureHunter. DS = DeepSeek Chat, GEM = Gemini-2.5-Flash-Lite, GPT = GPT-4o-mini. Bold indicates best result per column. 30 episodes per condition, Wilson 95% CIs in Appendix.

ments. The only difference is the memory management.

Analysis. Tables 15 and 16 show the same mission under Fade-Enriched History and Enriched History with identical seeds on TextWorld TreasureHunter very hard, Level 30. The room layout, object placement, and mission instructions are identical. The two conditions produce identical actions through Step 10, then diverge.

The divergence point. At Step 11, the Enriched History agent takes a wrong turn (go east instead of go south), entering an unproductive loop between Bathroom and Basement. This is a stochastic LLM decision that could occur under either condition. However, the consequences differ dramatically due to the structural difference in memory management.

FIFO displacement under Enriched History. With an enrichment window of $K=3$ (Table 3) and a buffer size of 16, the FIFO buffer can hold facts

covering approximately 48 steps of gameplay. At Step 11, the buffer contains only 4 facts, so F3 (“Retrieved TextWorld key from Lounge”) is safe. But the Bathroom–Basement loop generates a new navigation fact every 3 steps. By Step 50, the buffer has accumulated 17 facts and the oldest begin to be displaced. Around Step 53, F3 is pushed out of the top-16 FIFO. When the agent returns to the Laundromat at Step 55, F3 is no longer in the prompt. The agent sees the TextWorld gate but does not recall possessing the key and continues exploring until truncation at 80 steps. This is consistent with the aggregate diagnostics in Table 8: Enriched History with Gemini averages 12.7 steps per episode across all levels, but failed episodes on Level 30 run significantly longer as the agent exhausts the step budget without completing the mission.

Shield protection under Fade. The Fade-Enriched History agent follows the same path through Step 10 but does not enter the loop. Even if it had made the same wrong turn at Step 11, the

Condition	GoToObj			Open			Pickup			Unlock		
	DS	Gem	GPT	DS	Gem	GPT	DS	Gem	GPT	DS	Gem	GPT
Baseline	13	13	13	7	10	10	0	0	3	0	3	0
Reflection	13	13	13	0	10	17	0	0	3	0	0	10
Enriched	13	83	13	10	20	10	3	7	7	7	10	0
Trajectory	80	37	37	30	13	3	23	13	10	7	10	0
Enriched Hist.	70	63	50	23	30	17	17	10	10	10	13	13
Fade-Enr. Hist.	57	60	53	20	23	13	10	23	13	7	10	3

Table 7: Per-model success rates (%) on BabyAI. DS = DeepSeek Chat, GEM = Gemini 2.5 Flash Lite, GPT = GPT-4o-mini. 30 episodes per condition.

Model	Memory Type	Success Rate	Loop Rate	Avg. Steps (All)	Avg. Steps to Success
GPT	baseline	0.183	0.667	63.558	6.807
	reflection	0.208	0.642	61.875	7.312
	trajectory	0.733	0.058	15.442	8.201
	enriched	0.417	0.433	48.075	12.383
	enriched_history	0.717	0.058	16.675	9.129
	fade_enriched_history	0.733	0.092	18.375	8.743
GEM	baseline	0.208	0.758	64.450	14.292
	reflection	0.292	0.633	56.242	10.692
	trajectory	0.850	0.033	11.842	9.864
	enriched	0.592	0.500	36.233	17.133
	enriched_history	0.875	0.050	12.667	10.099
	fade_enriched_history	0.917	0.042	10.825	9.119
DS	baseline	0.350	0.617	55.342	12.863
	reflection	0.367	0.608	54.392	14.547
	trajectory	0.975	0.008	8.450	8.562
	enriched	0.708	0.325	29.650	12.025
	enriched_history	0.967	0.008	9.142	8.715
	fade_enriched_history	0.958	0.000	10.317	9.119

Table 8: Additional TextWorld diagnostic metrics averaged across all TextWorld levels for each backbone. DS = DeepSeek Chat, GEM = Gemini 2.5 Flash Lite, GPT = GPT-4o-mini. We report success rate, loop rate, average number of steps across all episodes, and average number of steps among successful episodes.

outcome would differ: F3 has a shield with trigger condition `locked.*(gate|chest)`. The shield caps decay at $\lambda_{\text{shield}} = 0.05$ (Table 3), ensuring F3 remains active regardless of how many new navigation facts enter the buffer. This structural advantage is reflected in the aggregate results: Fade with Gemini achieves 10.8 average steps (Table 8), 1.9 fewer than Enriched History, consistent with fewer episodes where the agent loses critical information and must recover.

Quantitative context. This divergence pattern explains the 6 percentage point gap between Fade and Enriched History on Level 30 with Gemini (83% vs. 77%, Table 6). The gap corresponds to approximately 2 out of 30 episodes where Fade succeeds and Enriched History fails. The FIFO buffer’s inability to distinguish task-critical facts (key possession) from transient navigation facts (room transitions) is the structural weakness that shields address. This advantage is specific to longer

episodes with key-lock dependencies: on CoinCollector (Table 12), which lacks such dependencies, Fade shows no advantage over Enriched History (80% vs. 83% with Gemini), confirming that the benefit requires temporal separation between information acquisition and use (Condition 2, Section 5.5).

Model-dependent effects. The same-seed comparison is shown for Gemini because it is the only backbone where Fade outperforms Enriched History on Level 30. With DeepSeek, both conditions achieve 87–93% (Table 6), leaving little room for divergence: DeepSeek reaches 97% with Trajectory alone, suggesting that raw context already suffices for this backbone (Condition 3). With GPT-4o-mini, Fade trails Enriched History (37% vs. 43%, Table 6), because the additional meta-reasoning calls introduce errors that outweigh the benefit of shielding. This trace therefore illustrates the sweet spot identified in Section 5.5: Fade helps most

Model	Memory Type	Success Rate	Loop Rate	Blocked Rate	Avg. Steps (All)	Avg. Steps to Success
GPT	baseline	0.067	0.875	0.615	72.075	12.250
	reflection	0.108	0.825	0.582	68.958	9.521
	trajectory	0.125	0.825	0.636	69.467	13.899
	enriched	0.075	0.742	0.674	71.300	13.528
	enriched_history	0.225	0.750	0.572	64.283	24.354
	fade_enriched_history	0.208	0.800	0.558	65.300	20.406
GEM	baseline	0.067	0.900	0.662	71.425	3.806
	reflection	0.058	0.908	0.710	72.050	3.208
	trajectory	0.183	0.717	0.378	65.625	18.580
	enriched	0.300	0.900	0.561	61.667	27.980
	enriched_history	0.292	0.700	0.458	61.758	26.155
	fade_enriched_history	0.292	0.750	0.425	61.500	27.456
DS	baseline	0.050	0.908	0.487	72.692	3.375
	reflection	0.033	0.883	0.493	73.958	2.750
	trajectory	0.350	0.750	0.344	59.758	33.018
	enriched	0.083	0.892	0.668	71.008	16.729
	enriched_history	0.300	0.817	0.350	62.133	30.100
	fade_enriched_history	0.233	0.758	0.344	63.758	24.662

Table 9: Additional BabyAI diagnostic metrics averaged across all BabyAI tasks for each backbone. DS = DeepSeek Chat, GEM = Gemini 2.5 Flash Lite, GPT = GPT-4o-mini. We report success rate, loop rate, blocked rate, average number of steps across all episodes, and average number of steps among successful episodes.

when the backbone is capable enough to execute meta-reasoning reliably (unlike GPT-4o-mini) but not so strong that raw context alone suffices (unlike DeepSeek).

Robustness vs. stochasticity. The wrong turn at Step 11 is not caused by memory. Both conditions have the same information available at that point. The difference is what happens after: Enriched History’s FIFO buffer is vulnerable to loop-generated fact displacement, while Fade’s shields make it robust to detours of any length. This is the core value proposition of dynamic memory management: not preventing mistakes, but preventing cascading information loss when mistakes occur.

M.4 GPT-4o-mini Conflict Classification Error

The following trace shows systematic conflict misclassification by GPT-4o-mini on TextWorld TreasureHunter very hard, Level 30 (fade_enriched_history, Seed 0, failure at 80 steps). The mission requires the agent to pick up a keycard, unlock a portal, navigate through 15+ rooms, and retrieve a passkey from the kitchen.

Episode overview. The episode has two distinct phases. In Phase 1 (Steps 0–9), the agent completes the first 10 mission steps correctly: picking up the keycard, unlocking and opening the portal, and navigating east and north through several rooms. In Phase 2 (Steps 10–79), the agent enters a navigation

loop between three rooms (Dish-Pit, Sauna, Basement), alternating between go north and go south without ever attempting go west, which is the next required mission step. The episode is truncated at 80 steps.

Label	Count	%
contradictory	68	50%
subsumed	43	32%
compatible	23	17%
subsumes	1	1%
Total	135	100%

Table 17: Conflict classification distribution for GPT-4o-mini, Episode 0, Seed 0. Half of all classifications are “contradictory.” Of these 68, 53 (78%) involve fact pairs that share at least one room name, describing the same area of the map.

Representative misclassifications. Table 18 shows three cases where GPT-4o-mini classifies near-identical facts as contradictory. In each case, the facts describe the same navigational situation with minimal differences in phrasing. GPT-4o-mini treats any variation in exit directions or room ordering as a logical contradiction, rather than recognizing these as compatible updates of the same state.

Qualitative comparison with Gemini. Gemini on the same architecture (Seed 0, Level 30) produces an even higher contradictory rate (34/40 = 85%), but with a critical qualitative difference.

Level	Memory Type	Success Rate (%)	Avg. Steps to Success	Avg. Steps (All)	Loop Rate
L10 (easy)	Baseline	23.3	4.43	59.97	0.733
	Reflection	26.7	7.50	58.03	0.733
	Enriched	56.7	13.00	37.23	0.467
	Trajectory	86.7	5.04	4.83	0.000
	Enriched History	80.0	5.33	7.57	0.000
	Fade-Enriched History	93.3	5.79	5.63	0.000
L15 (medium)	Baseline	16.7	4.80	67.47	0.567
	Reflection	13.3	3.75	67.53	0.600
	Enriched	56.7	12.88	41.97	0.300
	Trajectory	93.3	6.14	11.07	0.000
	Enriched History	100.0	8.17	8.17	0.000
	Fade-Enriched History	93.3	7.29	11.47	0.033
L20 (hard)	Baseline	16.7	13.60	66.33	0.767
	Reflection	23.3	15.00	62.23	0.700
	Enriched	30.0	14.22	57.67	0.500
	Trajectory	66.7	9.05	12.23	0.000
	Enriched History	63.3	7.63	13.77	0.067
	Fade-Enriched History	70.0	10.81	15.27	0.033
L30 (very hard)	Baseline	16.7	4.40	60.47	0.600
	Reflection	20.0	3.00	59.70	0.533
	Enriched	23.3	9.43	55.43	0.467
	Trajectory	46.7	12.57	33.63	0.233
	Enriched History	43.3	15.38	37.20	0.167
	Fade-Enriched History	36.7	11.09	41.13	0.300

Table 10: Detailed TextWorld TreasureHunter results for GPT-4o-mini. Success rate is reported over 30 episodes per level. Lower values are better for average steps and loop rate.

Task	baseline	reflection	trajectory	enriched	enriched_history	fade.enriched_history
GoToObj	83.33%	83.33%	61.11%	80.00%	56.67%	61.11%
Open	85.56%	86.67%	80.00%	82.22%	72.22%	76.67%
Pickup	93.33%	84.44%	78.89%	81.11%	85.56%	78.89%
Unlock	95.56%	94.44%	85.56%	94.44%	87.78%	91.11%

Table 11: Loop rates by BabyAI task and memory condition, averaged across all backbone LLMs.

Gemini’s contradictory classifications involve facts about genuinely different regions of the map (e.g., “moved west from Shower to Studio, then west to Cubicle” vs. “moved south from Bedroom to Cubicle, then east to Studio”). These describe different routes through different parts of the map; the classification is defensible. GPT-4o-mini’s contradictory classifications, by contrast, involve facts about the same three rooms with near-identical content. The distinction is not quantity of contradictions but quality: Gemini distinguishes genuinely incompatible states while GPT-4o-mini conflates surface-level variation with logical contradiction.

Impact on memory strength. Each false contradictory classification applies an exponential penalty averaging -0.240 to the older fact’s v_i . The strongest single penalty observed is -0.546 . Over the course of the episode, 9 facts are pushed below the dormant threshold ($v_i < 0.15$):

Step	v_i before	v_i after	Penalty
38	0.392	0.176	-0.216
38	0.423	0.172	-0.251
47	0.510	0.188	-0.322
56	0.173	0.064	-0.109
56	0.422	0.155	-0.267
56	0.458	0.169	-0.289
59	0.480	0.195	-0.285
71	0.392	0.159	-0.233
71	0.458	0.168	-0.290

Table 19: Facts pushed to dormancy by false contradictory classifications. Each row shows a single penalty event where a fact’s v_i dropped below the dormant threshold (0.15) or near it.

The cascading failure. The interaction between navigation failure and conflict misclassification creates a feedback loop:

1. GPT-4o-mini enters a navigation loop (Dish-Pit $\leftrightarrow$ Sauna $\leftrightarrow$ Basement) starting at Step 10.
2. Every 3 steps, a new navigation fact is encoded about these same rooms. The facts are

Condition	Success Rate (%)		
	DS	Gem	GPT
Baseline	10.0	3.3	6.7
Reflection	6.7	0.0	6.7
Enriched	30.0	20.0	10.0
Trajectory	100	83.3	96.7
Enriched Hist.	100	83.3	90.0
Fade-Enr. Hist.	100	80.0	86.7

Table 12: CoinCollector Level 10 success rates. DS = DeepSeek, Gem = Gemini 2.5 Flash Lite, GPT = GPT-4o-mini. Fade matches or slightly trails Enriched History on all backbones, consistent with the absence of temporal dependencies that would activate predictive tagging.

Step	Key Position	Action	LLM Reasoning
0	1 step right, 2 forward	turn right	“The green key is located 1 step to the right and 2 steps forward. I need to turn right to face the direction of the green key before moving forward.”
1	2 steps left, 1 forward	turn left	“The green key is now 2 steps left and 1 step forward from my current position. I need to turn left to face the direction of the green key.”
2	1 step right, 2 forward	turn right	“I need to turn right to face the direction of the green key, which is now 1 step right and 2 steps forward from my current position.”
3	2 steps left, 1 forward	turn left	“I need to turn left to align myself towards the green key, which is 2 steps left and 1 step forward from my current position.”
4	1 step right, 2 forward	turn right	“The green key is located 1 step to the right and 2 steps forward from my current position. I need to turn right to face the direction of the green key.”
5–63 <i>Identical oscillation continues. 0 forward attempts across all 64 steps.</i>			

Table 13: Trajectory condition, Episode 0, Seed 0. Mission: “go to the green key.” Backbone: GPT-4o-mini. The agent oscillates between turn right and turn left for 64 steps until truncation, never selecting go forward.

near-identical (“carrying keycard, navigated between sauna and dish-pit”).

3. GPT-4o-mini classifies the near-identical facts as contradictory instead of compatible or subsumed.
4. Each misclassification applies a penalty (mean -0.240) to the older fact’s v_i .
5. After 68 such penalties, critical facts (portal opened, route through Pantry/Laundry/Basement) fall below the dormant threshold and are excluded from the prompt.
6. Without mission progress information, the agent cannot recover.

The cumulative effect is visible in the dormant fact count: Steps 0–9: 0 dormant; Step 36: 2 dormant; Step 48: 5 dormant; Step 60: 13 dormant (half of all facts); Step 72: 20 dormant (more dormant than active). By Step 60, the agent has lost access to half its accumulated knowledge. By Step 72, the mission progress information (portal opened, keycard acquired, route taken) has completely dis-

appeared from active memory.

Quantitative context. GPT-4o-mini exhibits a substantially higher rate of harmful contradiction misclassifications in our trace analysis than the stronger backbones, particularly when repeated navigation facts about the same rooms are repeatedly marked as contradictory. The high false-positive rate explains why Fade underperforms Enriched History with GPT-4o-mini on L30 (37% vs. 43%, Table 6): the meta-reasoning calls actively degrade memory quality. Enriched History’s FIFO buffer, which never actively suppresses facts, avoids this failure mode entirely. Gemini, despite a higher per-episode contradictory rate in some episodes, classifies qualitatively different content and navigates efficiently enough that the penalties do not cascade.

This supports Condition 3 (Section 5.5): the Fade pipeline requires a backbone capable enough to execute meta-reasoning reliably. GPT-4o-mini falls below this threshold, producing cascading

Step	Key Position	Action	LLM Reasoning
0	2 steps left, 2 forward	turn left	“The yellow key is located 2 steps left and 2 steps forward from my current position. I need to turn left to face the direction of the yellow key.”
1	2 steps right, 2 forward	turn right	“The yellow key is now 2 steps to the right and 2 steps forward. I need to turn right to face the direction of the yellow key.”
2	2 steps left, 2 forward	turn left	“I need to turn left to align myself with the yellow key, which is now 2 steps left and 2 steps forward from my current position.”
3	2 steps right, 2 forward	turn right	“The yellow key is located 2 steps right and 2 steps forward from my current position. I need to turn right to align myself with the key’s direction.”
4–63 <i>Identical oscillation continues. 0 forward attempts across all 64 steps.</i>			

Table 14: Enriched History condition, Episode 1, Seed 1. Mission: “go to the yellow key.” Backbone: GPT-4o-mini. Despite access to a summary of prior actions, the agent exhibits the same oscillation pattern as the Trajectory condition (Table 13).

classification errors that compound over long episodes. The failure is not simply that GPT-4o-mini makes more errors, but that the errors are systematically wrong in a way that degrades memory precisely when it is most needed.

N Loop Detector

Behavioral loops are detected via two heuristics:

Streak detection. A loop is registered when ≥ 5 consecutive steps share the same (observation, action) pair. Observations are normalized by lower-casing and collapsing whitespace before comparison.

Oscillation detection. A loop is registered when ≥ 6 consecutive actions follow an alternating A-B-A-B pattern (e.g., “go north”, “go south”, “go north”, “go south”, ...).

Both thresholds were chosen based on manual inspection of 50 episodes across environments.

O Metric Definitions

Success Rate (SR). $SR = \frac{1}{N} \sum_{i=1}^N s_i$ where $s_i = 1$ if the episode terminated with reward > 0 , and $s_i = 0$ otherwise.

Average Steps to Success (AS). $AS = \frac{\sum_{i:s_i=1} T_i}{\sum_i s_i}$ where T_i is the number of environment steps in episode i . Only successful episodes are included.

Loop Rate (LR). $LR = \frac{1}{N} \sum_{i=1}^N l_i$ where $l_i = 1$ if a loop was detected (see above).

Blocked Forward Rate (BFR). BabyAI only.
 $BFR = \frac{\text{blocked forward steps}}{\text{total forward attempts}}$. A forward attempt is

blocked when the agent’s grid position does not change after executing “go forward.”

Conflict Resolution Optimization To improve efficiency, we prepend a deterministic fast path to the FadeMem conflict resolution pipeline. This stage uses regex pattern matching to detect state-change contradictions (e.g., locked/unlocked) via shared entity tokens. Obvious conflicts are resolved immediately and only ambiguous cases proceed to the LLM-guided classification described in (Wei et al., 2026).

Step	Room	Action	Memory Event	v_i (F3)
0	Laundromat	take American latchkey		
1	Laundromat	unlock American gate		
2	Laundromat	open American gate	F1: “American gate opened.” Shield set. Trigger: locked.* (gate door)	
3	Cubicle	go west		
4	Bedchamber	go west		
5	Bedchamber	go north	F2: “Navigated west through Cubicle to Bedchamber.”	
6	Spare Room	go north		
7	Lounge	take TextWorld key	F3: “Retrieved TextWorld key from Lounge.” $v_i = 0.98$. Shield set. Trigger: locked.* (gate chest)	0.98
8	Lounge	go east		0.97
9	Shower	go south		0.96
10	Studio	go west	Trigger fired on F3. $v_i: 0.95 \rightarrow 1.00$ (capped).	1.00
11	Spare Room	go south		0.98
12	Bedchamber	go east		0.97
13	Cubicle	go east		0.95
14	Laundromat	unlock TW gate	F3 in prompt ($v_i = 0.93$). Agent remembers the key.	0.93
15	Laundromat	open TextWorld gate		
16	Laundromat	go east		
17	Workshop	take key	Mission complete. Reward = 1.0	

Table 15: Fade-Enriched History, Seed 1. Success in 18 steps. The shield on F3 fires at Step 10 and protects the key fact throughout navigation.

Step	Room	Action	FIFO Buffer State	Facts
0	Laundromat	take American latchkey		
1	Laundromat	unlock American gate		
2	Laundromat	open American gate	F1: “American gate opened.”	1
3–6	<i>(identical to Fade)</i>		F1, F2	2
7	Lounge	take TextWorld key	F1, F2, F3: “Retrieved TW key.”	3
8–10	<i>(identical to Fade)</i>		F1, F2, F3	3
11	Spare Room	go east (wrong turn)	F1, F2, F3, F4	4
12–14	Shower → Basement	<i>(detour)</i>	F1–F3, F4, F5	5
15–17	Bathroom ↔ Basement	<i>(loop begins)</i>	F1–F3, F4–F6	6
18–20	Basement ↔ Bathroom	<i>(loop continues)</i>	F1–F3, F4–F7	7
21–35	<i>(loop continues, 15 steps)</i>		F1–F3, F4–F12	12
36–50	<i>(loop continues, 15 steps)</i>		F4–F17. F1 displaced.	16
51–53	<i>(loop continues)</i>		F5–F18. F3 displaced.	16
55	Laundromat	<i>(agent returns)</i>	F3 not in prompt. Agent sees TW gate but does not recall the key.	16
56–80	<i>(exploration, failed attempts)</i>		FAIL (truncated/stopped at 80 steps)	

Table 16: Enriched History, same Seed 1. Failure at 80 steps. After a wrong turn at Step 11, the agent enters a navigation loop. By Step 53, F3 (“Retrieved TextWorld key”) is displaced from the 16-fact FIFO buffer by loop-generated navigation facts.

Old Fact	New Fact	GPT	Correct
“Carrying keycard, navigated between sauna and dish-pit, unblocked exit north from sauna.”	“Carrying keycard, navigated between sauna and dish-pit, heading north from dish-pit back to sauna.”	contr.	compat.
“Carrying keycard, navigated between dish-pit and sauna, exit south from dish-pit.”	“Carrying keycard, navigated between dish-pit and sauna, exit south from sauna.”	contr.	compat.
“Carrying keycard, navigated between sauna, dish-pit, and basement, exit north.”	“Carrying keycard, navigated between dish-pit and sauna, exits south and west.”	contr.	subsumes

Table 18: Representative conflict misclassifications. “contr.” = contradictory, “compat.” = compatible. Facts are abbreviated for space; full text in the trace data.

Step	Room	Action	Memory Event	v_i
0	Laundry Place	go east		
1	Bedroom	take type 3 latchkey		
2	Bedroom	look	Fact created: "Agent picked up type 3 latchkey in bedroom." Shield set. Trigger: locked.* (chest door passageway)	0.967
3	Bedroom	go west		0.952
4	Laundry Place	open passageway		0.941
5	Laundry Place	go north		0.870
6	Kitchen	examine counter	Counter is empty.	0.801
7	Kitchen	go east		0.737
8	Salon	examine shelf	Nothing useful.	0.678
9	Salon	go north		0.624
10	Cubicle	examine desk	Nothing useful.	0.574
11	Cubicle	go west		0.528
12	Pantry	examine toolbox	Toolbox is closed.	0.486
13	Pantry	go south		0.447
14	Chamber	examine box	Box is empty.	0.411
15	Chamber	go east		0.378
16	Scullery	go south		0.348
17	Workshop	examine workbench	Nothing useful.	0.320
18	Workshop	go west		0.294
19	Cellar	go south		0.271
20	Cellar		Trigger fired. Observation: "locked type 3 chest." Shield consumed. v_i : 0.271 $\rightarrow$ 0.471 (+0.2 boost). Fact stays active.	0.471
21	Cellar	unlock type 3 chest	Agent remembers latchkey because fact is in prompt.	0.460
22	Cellar	open type 3 chest	Reveals latchkey. Fact created: "Latchkey taken from chest." Shield set. Trigger: locked.* (door passage)	0.449 / 0.967
23	Cellar	take latchkey		0.438 / 0.952
<i>Exploration through multiple rooms (Steps 24–35). Conflict resolutions; older facts lose v_i.</i>				
36			Reactivation: dormant navigation fact reactivated. v_i : 0.147 $\rightarrow$ 0.447	0.447
37			Trigger fired. Latchkey fact held at $v_i = 1.0$	1.000
<i>Continued exploration (Steps 38–42).</i>				
43	Closet	take pair of pants	Trigger expired. Mission complete. Reward = 1.0	

Table 20: Predictive Tagging trace. TextWorld TreasureHunter very hard, Level 30, Gemini 2.5 Flash Lite, fade_enriched_history. The v_i column tracks the strength of the fact "Agent picked up type 3 latchkey in bedroom" unless otherwise noted. The active threshold is 0.15.